

BREACHKEEP FACULTY CURRICULUM

The Defender's Compass

A Student Handbook to Cybersecurity Foundations: Exploring Networks, the Web, and Secure Engineering Through the BreachKeep Dungeons

Course: Introduction to Applied Cybersecurity (eLabs)

Focus Dungeons: Signals (Network) • Trading Post (Web) • The Forge (Secure Coding)

Target Audience: Beginners • Aspiring Ethical Hackers • Software Craftsmen

A Note from Your Teacher: Welcome to the Keep!

Dear Cadet,

Welcome to cybersecurity. When people first hear about hacking, their minds often leap to movie clichés: darkened rooms, neon green text cascading across black monitors, and furious typing that miraculously breaks into satellite systems in twelve seconds flat. The reality is both more grounded and far more exciting.

Cybersecurity is not digital witchcraft. It is the art and science of understanding how systems communicate, discovering the hidden assumptions that developers and architects make, and observing what happens when those assumptions break down. Every vulnerability you will encounter in BreachKeep was not created by malice, but by a very human developer who was tired, rushed, or unaware of how their code would behave under unexpected pressure.

The Teacher's Mental Model: Think of a secure system like a masterfully constructed building. The network is the plumbing and electrical wiring—the conduits through which all energy and information flow. The web application is the public reception desk and front lobby—where the world comes to interact with the building's services. And secure coding is the architectural blueprint and structural engineering—the careful design that ensures a single unlocked window doesn't cause the entire building to collapse.

In this handbook, we will walk through three core dungeons of BreachKeep together:

1. **The Network Dungeon ("Signals"):** Learning how machines whisper across physical and virtual wires, how ports act as apartment doors, and how packets can be intercepted, read, and shaped.
2. **The Web Dungeon ("The Trading Post"):** Exploring the modern browser and server relationship, understanding stateless HTTP, and breaking common misconceptions about client-side safety.
3. **The Secure Coding Dungeon ("The Forge"):** Transitioning from an attacker who breaks systems to a master craftsman who builds resilient, unassailable software that protects

users and data.

Do not be afraid to make mistakes. A broken script, an error 500, or a denied connection is not a failure; it is data. It is the system telling you exactly how it thinks. Let's take our first steps together.

Table of Contents

Part I: The Network Dungeon — Signals

Chapter 1: The Anatomy of a Connection

Challenges 1–10: Ports, Scans, Banners, Packets, DNS, HTTP, Protocol Analysis, Firewalls, Forensics, & Pivoting

Part II: The Web Dungeon — The Trading Post

Chapter 2: The Web Architecture & The Illusion of Client Trust

Challenges 1–10: Recon, DevTools, Cookies, Client Trust, IDOR, SQLi, Reflected XSS, Headers, Stored XSS, & Exploit Chains

Part III: The Secure Coding Dungeon — The Forge

Chapter 3: The Defensive Engineer's Mindset

Challenges 1–8: Prepared Statements, Sanitization, Ownership Checks, Input Validation, CSP, Rate Limiting, Secret Management, & Full Code Audits

PART I • DUNGEON 3

The Network Dungeon: Whispers on the Wire

Chapter 1: How Computers Talk Across the Void

Before we touch a terminal, let's understand what a network actually is. A network is simply a group of computers agreed upon a common set of rules (protocols) to exchange messages. If you send a physical letter to a friend, you write an address (street name and number), place a stamp on it, and hand it to the postal service. The postal service doesn't care what is written inside the letter; its only job is reading the envelope and delivering it to the right destination.

Real-World Analogy: IP Addresses & Ports

Imagine an IP address is the physical street address of a massive apartment complex (e.g., `10.200.1.5`). If you only provide the street address, the mail carrier can reach the building lobby, but they don't know who receives the letter.

Port numbers are the apartment numbers (e.g., Apartment 80, Apartment 22, Apartment 53). When a letter arrives at the building, the apartment number tells the system which specific tenant (service or daemon) the packet is addressed to.

In Linux networking, services listen on specific ports. Standard port conventions include:

- **Port 22 (SSH):** The secure shell for remote administration.
- **Port 53 (DNS):** The domain name system phonebook.
- **Port 80 (HTTP):** Plaintext web traffic.
- **Port 443 (HTTPS):** Encrypted web traffic over TLS.

Now, let's examine the 10 challenges in the Signals dungeon and discover how attackers and defenders navigate this landscape.

Challenge 1: What's Listening Behind the Door? (network-ports)

Mandatory • Socket Enumeration

The Story & Context: You log into a remote host. You are told that several services are bound and listening on the machine, but you don't know which port hosts the authentic keep service and which ones are merely decoys or telemetry probes.

Teacher's Lesson — Socket Investigation:

How do you ask a Linux kernel "What programs are currently waiting for incoming network connections?" We use socket statistics tools: `ss` or `netstat`.

The command `ss -tulpn` breaks down as:

- `-t` : Display **TCP** sockets.
- `-u` : Display **UDP** sockets.
- `-l` : Show only **listening** sockets (waiting for a connection).
- `-p` : Show the **process** using the socket (requires root).
- `-n` : Show **numeric** port numbers instead of attempting to resolve names.

The Hands-on Path:

```
student@signals:~$ ss -ltn
State  Recv-Q  Send-Q  Local Address:Port  Peer Address:Port
LISTEN  0        128     127.0.0.1:2451      0.0.0.0:*
LISTEN  0        128     127.0.0.1:3120      0.0.0.0:*
LISTEN  0        128     127.0.0.1:5678      0.0.0.0:*
LISTEN  0        128     127.0.0.1:8290      0.0.0.0:*
```

Once you see the open ports, how do you talk to them? You use `netcat` (`nc`), known in cybersecurity as the Swiss Army Knife of networking. Running `nc 127.0.0.1 5678` opens a raw TCP connection, allowing you to read the server's immediate greeting.

Challenge 2: Mapping the Dark House (network-scan)

Mandatory • Port Scanning &
Nmap

The Concept & Intuition: In the real world, you cannot log into the target machine to run `ss -ltn`. You are on the outside looking in. Port scanning is like walking down the hallway of an office building at night and gently turning every door handle to see which offices are unlocked.

The TCP Three-Way Handshake:

TCP connections are polite conversations that require three steps:

1. **SYN (Synchronize):** "Hello! May I talk to you?" (Sent by client).
2. **SYN-ACK (Synchronize-Acknowledge):** "Yes, I hear you and I'm open!" (Sent by server if port is open).
3. **ACK (Acknowledge):** "Great, let's begin!" (Sent by client).

If the port is closed, the server replies with **RST (Reset):** "Go away, nobody lives here."

Using Nmap: To map our target at `127.0.0.2`, students use `nmap` :

```
student@signals:~$ nmap -sT -p 1-10000 127.0.0.2
Starting Nmap ( https://nmap.org )
Nmap scan report for 127.0.0.2
PORT      STATE SERVICE
2100/tcp  open  echo
3400/tcp  open  daytime
7890/tcp  open  unknown
```

Students learn that scanning reveals services without ever touching the server's console. Connecting to port 7890 exposes the hidden vault service.

Challenge 3: Reading the Welcome Mat (network-banner)

Fingerprinting • Banner
Grabbing

The Concept: Knowing a port is open is only half the battle; what software is running behind it? Many network daemons eagerly announce their software name, version, and operating system the moment a client connects. This is called a **service banner**.

Why Do Services Announce Themselves?

Historically, protocol specifications (like FTP and SMTP) encouraged software to identify itself for administrative diagnostics. In modern security, however, broadcasting `Apache 2.4.49` or `OpenSSH 7.2p2` tells an attacker the exact patch level and known CVE vulnerabilities of your server!

Students use `nc -nv <ip> <port>` or `curl -I` to capture these banners and identify the specific service version harboring credentials.

Challenge 4: The Transparent Postal System (network-capture)

Packet Sniffing • Tshark & Wireshark

The Concept: When data travels over a network wire or Wi-Fi channel, it is broken into discrete units called **packets**. If communication is not encrypted (e.g., plain HTTP, FTP, or Telnet), anyone positioned between the sender and receiver can read every single byte.

Postcards vs. Wax-Sealed Letters:

Plain HTTP is like writing a message on a postcard. Every mail carrier, sorting clerk, and bystander who handles the postcard can read your message without tearing an envelope. HTTPS (TLS) is like placing the message inside an indestructible, tamper-evident steel safe that only the intended recipient can unlock.

The Hands-on Path: Students inspect a packet capture file (`capture.pcap`) using command-line Wireshark (`tshark`):

```
student@signals:~$ tshark -r capture.pcap -Y "http.request or http.response"
 1  0.000000  10.0.0.2 -> 10.0.0.1  HTTP GET /login HTTP/1.1
 2  0.004512  10.0.0.1 -> 10.0.0.2  HTTP HTTP/1.1 200 OK (text/html)
 3  0.012930  10.0.0.2 -> 10.0.0.1  HTTP POST /login HTTP/1.1 (application
```

By inspecting packet #3 with `tshark -r capture.pcap -T fields -e text`, students see raw passwords sent across the wire in plaintext, teaching the non-negotiable necessity of TLS.

Challenge 5: The Internet's Phonebook (network-dns)

DNS Reconnaissance • Dig & TXT
Records

The Concept: Humans are terrible at remembering numbers like `142.250.190.46`, but great at remembering names like `google.com`. The Domain Name System (DNS) is the global directory that translates names to IP addresses.

DNS supports many record types:

- **A Records:** Maps a domain to an IPv4 address.
- **MX Records:** Mail Exchangers that receive email.
- **TXT Records:** Arbitrary text metadata, often used for domain verification, SPF email safety, and internal administrator notes.

Students use the `dig` tool to query a local DNS server and uncover hidden configuration strings stored inside TXT records:

```
student@signals:~$ dig @127.0.0.1 -p 53 internal.keep.lan TXT +short  
"ARCHIVE::dns_z0n3_tr4nsf3r_....:"
```

Challenge 6: The Language of the Web (network-http)

Protocol Mechanics • Curl &
Headers

The Concept: Web browsers hide the raw HTTP protocol behind pretty graphics. In cybersecurity, we must strip away the browser and speak directly to web servers using terminal tools like `curl`.

Anatomy of an HTTP Request:

An HTTP request is pure text containing three main elements:

1. **Request Line:** The method and path (e.g., `GET /index.html HTTP/1.1`).
2. **Headers:** Key-value metadata (e.g., `User-Agent: curl/7.88.1`, `X-Custom-Auth: secret`).
3. **Body:** The payload (used in POST and PUT requests).

Students use `curl -i -H "X-Warden-Token: secret" http://127.0.0.1:8080/api` to manually inject required authentication headers, experiencing firsthand how HTTP requests are structured.

Challenge 7: Speaking Alien Dialects (network-protocol-id)

Protocol Triage • Text vs. Binary

The Concept: Not every protocol on a network is human-readable HTTP or SMTP. Many proprietary, industrial, or gaming protocols communicate in raw binary packets. When an unknown port opens, how do you identify what language it speaks?

Students connect to mystery endpoints and observe how they respond:

- Does it print ASCII text? (e.g., `220 Service Ready` → SMTP/FTP).
- Does it emit binary bytes? (e.g., `PNG` or `??` → Gzip/Zlib).
- Does it echo back input or expect an immediate binary handshake?

This develops intuitive diagnostic triage skills required during real-world penetration tests.

Challenge 8: The Bouncer at the Gate (network-firewall)

Defensive Rules • iptables & Ingress Filtering

The Concept: A firewall is a network traffic filter. The Linux kernel includes a built-in packet filtering framework called `netfilter`, managed via `iptables` or `nftables`.

The Default-Deny Golden Rule:

A properly configured firewall should follow **Default Deny**: "Block all incoming traffic by default, and explicitly allow only the specific ports and IP addresses that are strictly necessary."

Students inspect firewall tables using `iptables -L -n -v`, identifying misconfigured rules where specific ports or subnets were inadvertently left open to unauthorized networks.

Challenge 9: Digital Archaeology (network-pcap-forensics)

Forensics • Stream
Reassembly

The Concept: Large files (like images, archives, or executable binaries) cannot fit in a single 1500-byte network packet. TCP breaks files into dozens or thousands of segments, transmits them out of order, and reassembles them at the receiver.

Students learn how network forensic analysts extract transmitted files from recorded PCAP streams:

```
tshark -r stream.pcap --export-objects "http,./extracted_files"
```

Carving files out of packet streams demonstrates how evidence is recovered during incident response investigations.

Challenge 10: Stepping Across Islands (network-pivot)

Advanced • Multi-Hop Network
Pivoting

The Concept: In enterprise security, sensitive internal databases are never connected directly to the public internet. They sit in deep internal network segments. When an attacker compromises an external-facing server (a jump host or bastion), they use that host as a bridge to reach internal networks. This technique is known as **network pivoting**.

The Island Bridge Analogy:

Imagine you stand on Island A. Island C is miles away across deep water, and you cannot jump to it. But Island B sits in the middle with a bridge connecting to Island A, and another bridge connecting to Island C. If you walk across the bridge to Island B, you can now reach Island C!

Students configure multi-hop SSH tunnels (`ssh -D 1080`) or proxy chains to route traffic through compromised intermediate hosts, unlocking deeply protected internal nodes.

PART II • DUNGEON 4

The Web Dungeon: Breaking the Browser's Trust

Chapter 2: The Modern Web & The Illusion of Client Trust

The web is built on a fundamental division of labor: the **Client** (the student's web browser) and the **Server** (the backend computer hosting databases and business logic). The browser renders HTML, applies CSS styling, and executes JavaScript to create snappy user interfaces.

However, this division creates the single most common vulnerability category in all of computer science: **The Fallacy of Client Trust**.

The First Law of Web Security:

Never trust the client. The client machine is owned and operated by the user. An attacker has full control over their browser, can inspect every line of front-end JavaScript, can modify every HTTP request, and can tamper with every cookie, parameter, and header before sending it to the server.

Challenge 1: Read the Map (web-recon)

Mandatory • Information Disclosure

The Concept: Search engines like Google crawl the web by following links. Webmasters use a text file at the root of their website called `/robots.txt` to tell automated crawlers: *"Please do not index these private folders."*

The Irony of robots.txt:

To tell a polite crawler what not to look at, the administrator must list all the secret paths in plain text! An attacker looks at `/robots.txt` first, treating it as an official treasure map of hidden endpoints.

```
User-agent: *  
Disallow: /keep-backup/  
Disallow: /admin-console/
```

Visiting `/keep-backup` immediately reveals exposed development notes and backup indices that were never meant for public eyes.

Challenge 2: Under the Hood of Your Browser (web-devtools)

Browser Inspection • Custom
HTTP Headers

The Concept: Most users only look at the graphical elements rendered on their screen. But web pages communicate dozens of metadata fields in the background via **HTTP Response Headers**.

Students open their browser Developer Tools (F12) → **Network Tab**, refresh the page, and inspect the raw response headers for `/account`. There, hiding in plain sight, sits a header like:

```
HTTP/1.1 200 OK
Content-Type: text/plain
X-Keep-Token: S0VZX34wcnRfc2Nhbm5lc19ubzBi...
```

Decoding the Base64 token reveals the flag, teaching students that web security extends far beyond what is visible on the rendered page.

Challenge 3: The VIP Wristband Problem (web-cookie-trust)

Session Tampering •
Insecure Cookies

The Concept: HTTP is **stateless**. When you click from page A to page B, the server forgets who you are unless you provide a token proving your identity. The most common mechanism for this is a **cookie**.

The Waterpark Wristband Analogy:

Imagine a waterpark hands you a paper wristband that says `role=guest` written in blue ballpoint pen. If the lifeguard at the VIP slide simply glances at the wristband and lets anyone through who has `role=admin` written on it, what stops a guest from crossing out "guest" with a marker and writing "admin"?

In this challenge, the application checks:

```
if (req.cookies.role === 'admin') {  
  return res.send("Welcome Warden: " + flag);  
}
```

By opening DevTools → Application → Cookies and editing their cookie from `role=user` to `role=admin`, students walk right past the gate, illustrating why authorization must be cryptographically signed (like HMAC-signed JWTs or secure session stores).

Challenge 4: The Honor System at Checkout (web-client-trust)

Parameter Tampering • Business Logic Bypass

The Concept: An online store displays an item with a price of \$50. The HTML form includes an input field: `<input type="hidden" name="price" value="50">`.

Why Do Developers Do This?

Front-end developers often think: "*The user cannot change this because it's a hidden field or disabled input.*" But the user is sending the HTTP POST request! An attacker can open `curl` or Burp Suite and change the request body to `price=0` or `price=-50`.

If the server accepts whatever price the client submits without looking up the true cost in its database, the user gets items for free. Students learn that all business logic and calculations must live strictly on the server.

Challenge 5: Peeking in Someone Else's Locker (web-idor)

Broken Access Control •
IDOR

The Concept: Insecure Direct Object References (IDOR) occur when an application takes an ID supplied by the user (like an order number, user ID, or invoice ID) and retrieves the record from the database without verifying if the currently logged-in user actually owns that record.

```
// VULNERABLE IDOR IMPLEMENTATION
app.get('/order', (req, res) => {
  const order = db.get("SELECT * FROM orders WHERE id = ?", req.query.id);
  res.json(order); // Oops! We never checked who owns this order!
});
```

A student logs in, views their own receipt at `/order?id=1042`, changes the number in their URL bar to `/order?id=1337`, and immediately views the administrator's private purchase records. This teaches the importance of checking session identity against resource ownership on every single access.

Challenge 6: The Waiter and the Kitchen (web-sqli)

Database Exploitation • SQL Injection

The Concept: Databases store structured information. When an application queries a database, it speaks SQL (Structured Query Language). A classic login query looks like:

```
SELECT * FROM users WHERE username = 'alice' AND password = 'secretpassword';
```

The Restaurant Ticket Analogy:

Imagine a waiter writes your order on an order ticket for the kitchen: *"Table 4 wants: [CUSTOMER INPUT]"*.

If a customer says: *"A steak, AND ALSO GIVE TABLE 4 ALL MONEY IN THE CASH REGISTER"*, and the kitchen blindly obeys the entire handwritten ticket as an executive instruction, that is SQL Injection!

If a developer concatenates user input with strings: `"SELECT * FROM users WHERE username = '" + user + "'"`, an attacker inputs:

```
admin' OR '1'='1' --
```

The resulting query becomes: `SELECT * FROM users WHERE username = 'admin' OR '1'='1' --' ...`. Because `'1'='1'` is always true, and `--` comments out the password verification, the database logs the attacker in as administrator without a password!

Challenge 7: Echoing Poison (web-reflected-xss)

Client-Side Injection • Reflected XSS

The Concept: Cross-Site Scripting (XSS) occurs when an application takes untrusted user input and embeds it into the HTML page without sanitizing or escaping it. The victim's browser cannot tell the difference between legitimate code written by the website author and malicious code injected by an attacker.

In `web-reflected-xss`, the search box reflects whatever you typed:

```
<h1>Results for: <script>alert(1)</script></h1>
```

Because the angle brackets `< >` are rendered directly into the HTML document object model (DOM), the browser executes the script inside the user's active session, allowing an attacker to steal session cookies or perform unauthorized actions.

Challenge 8: The Return Address Lie (web-headers)

Access Control Bypass • Header Forgery

The Concept: Some developers try to restrict administrative portals by checking the `X-Forwarded-For` or `Referer` headers, assuming requests claiming to come from `127.0.0.1` must be internal and trustworthy.

Students learn that **HTTP request headers are completely controlled by the client**. By supplying `curl -H "X-Forwarded-For: 127.0.0.1"` or custom origin headers, students demonstrate why security boundaries must never rely on unverified client-asserted metadata.

Challenge 9: A Time Bomb on the Bulletin Board (web-stored-xss)

Persistent Injection •
Stored XSS

The Concept: Reflected XSS requires tricking a victim into clicking a crafted link. **Stored XSS** is much more dangerous: the malicious script is submitted to a persistent storage location (like a guestbook, forum post, or profile bio).

When any other user (including the platform administrator) visits the guestbook, the server pulls the malicious comment out of the database and delivers it to their browser, executing silently inside their session without any link clicking required.

Challenge 10: The Domino Effect (web-chain)

Advanced • Multi-Stage Exploit
Chaining

The Concept: In real-world security assessments, a single small vulnerability rarely brings down an entire fortress. Attackers achieve critical compromises by **chaining** multiple low-severity findings together.

The 3-Step Exploit Chain:

1. **Step 1 (Reconnaissance):** Inspect `/robots.txt` to discover the hidden administrative endpoint: `/keep-admin`.
2. **Step 2 (Access Control Violation):** Visit `/keep-admin/vault` and receive a 403 Forbidden error stating "admin role required".
3. **Step 3 (Session Tampering):** Set your browser cookie to `role=admin` and refresh, unlocking the inner vault!

This challenge brings together all lessons learned in the Web Dungeon, illustrating how small informational leaks pave the way for total administrative compromise.

PART III • DUNGEON 5

The Secure Coding Dungeon: The Art of Defensive Engineering

Chapter 3: The Defensive Engineer's Mindset

Breaking a vulnerable application is fun, eye-opening, and thrilling. But anyone can knock down a wall with a sledgehammer; it takes true mastery, deep engineering discipline, and craft to design a wall that cannot be knocked down in the first place.

In **The Forge**, students put on the hardhat of a software engineer. You are handed the actual source code of the vulnerable web application and given an automated test suite. Your mission is twofold:

1. **Neutralize the exploit:** Make sure malicious inputs and attacks fail completely.
2. **Keep the application working:** Ensure that legitimate users can still log in, view their orders, and use the system without disruption.

Challenge 1: Separating Code from Data (secure-sqli)

Defensive SQL • Prepared Statements

The Flawed Attempt (Blacklisting): Beginners often try to fix SQL injection by writing filter blacklists:

```
// FLAWED FIX — DO NOT DO THIS!  
if (username.includes("'")) return res.status(401).send("No quotes allowed!");
```

Why does this fail? First, an attacker can often bypass filters using alternative encodings. Second, what happens when a real user named **O'Brien** tries to log into your service? The blacklist breaks legitimate business functionality!

The Secure Solution — Parameterized Queries:

Instead of pasting strings together, use **Prepared Statements**. A prepared statement pre-compiles the SQL query structure in the database engine first. The user input is then sent separately as pure data parameters. Even if the user enters `' OR 1=1 --`, the database treats it as a literal string value for a username, never as executable SQL logic!

```
// SECURE IMPLEMENTATION  
const stmt = db.prepare('SELECT * FROM users WHERE username = ? AND password = ?  
const row = stmt.get(username, password);
```

Challenge 2: Teaching the Browser What is Safe (secure-xss)

Output Encoding • HTML
Entity Escaping

The Flawed Attempt (Tag Stripping):

```
// FLAWED FIX
const sanitized = input.replace(/<script>/gi, "");
```

Stripping `<script>` tags fails immediately because modern HTML supports dozens of ways to execute JavaScript without script tags: ``, `<svg onload="alert(1)">`, or `<body onload="...">`.

The Secure Solution — Context-Aware HTML Encoding:

Whenever user data is rendered into an HTML document, convert special characters into their safe HTML entity equivalents:

- `&` → `&`
- `<` → `<`
- `>` → `>`
- `"` → `"`
- `'` → `'`

When the browser sees `<script>`, it renders the characters as harmless readable text on screen without ever executing them as code.

Challenge 3: Checking the ID at the Counter (secure-idor)

Authorization • Session
Ownership Binding

The Flawed Fix: Hardcoding checks for specific IDs (e.g., `if (id === 1337) return 403`) fails as soon as a new record is inserted.

The Secure Solution — Scope Queries to Session Identity:

Never query the database solely by the client's requested ID. Always bind the query to the authenticated user stored in the session cookie:

```
// SECURE IMPLEMENTATION
const currentUser = req.session.user; // Authenticated server-side identity
const order = db.prepare('SELECT * FROM orders WHERE id = ? AND owner = ?')
    .get(req.query.id, currentUser);
if (!order) return res.status(404).send("Order not found or unauthorized");
```

Challenge 4: Trust, But Verify on the Server (secure-client)

Input Validation • Server-Side
Enforcement

The Concept: To fix client-side price tampering, the server must validate all incoming data against strict business rules:

- Is the value a valid finite number?
- Is the value strictly greater than zero?
- Better yet: Look up the authentic price in the product catalog database using a product ID rather than trusting a client-supplied price tag!

```
// SECURE IMPLEMENTATION
const price = Number(req.body.price);
if (isNaN(price) || price <= 0) {
  return res.status(400).send("Invalid order amount");
}
```

Challenge 5: Armoring the Browser (secure-headers)

Defense-in-Depth • Security
Headers & CSP

The Concept: Security headers instruct the user's browser to activate its built-in security defenses.

Header	What It Does
Content-Security-Policy (CSP)	Restricts where scripts, styles, and images can be loaded from. Prevents unauthorized inline script execution.
X-Frame-Options: DENY	Stops clickjacking by preventing other sites from embedding your page in an iframe.
X-Content-Type-Options: nosniff	Prevents the browser from misinterpreting a text or image file as an executable script.

Students implement standard headers using modern middleware like `helmet` .

Challenge 6: Calming the Storm (secure-rate-limit)

Anti-Abuse • Token Buckets & 429 Limiting

The Concept: Computers can submit thousands of passwords or requests per second. To protect servers from denial-of-service and brute-force attacks, we implement **Rate Limiting**.

The Bouncer's Clicker:

A rate limiter tracks requests per IP address or account over a sliding time window (e.g., maximum 5 attempts per 60 seconds). Once the limit is reached, the server responds with **HTTP 429 Too Many Requests** and a `Retry-After` header, resetting only after the window expires.

Challenge 7: Keeping the Keys in the Safe (secure-hide-secret)

Secrets Management • Environment Decoupling

The Mistake: Storing API keys, database passwords, or cryptographic secrets directly in source code files.

Why Hardcoding Keys Is Fatal:

Code gets committed to Git repositories, pushed to GitHub, shared with teammates, and included in backups. Once an API key is committed to source control, it is practically public knowledge.

The Secure Pattern: Store all credentials in external environment variables (accessed via `process.env.API_KEY`) that are injected into the container at runtime and excluded from source repositories and public configuration endpoints.

Challenge 8: The Master Builder's Final Inspection (secure-full-review)

Full-Stack Hardening •
Comprehensive Audit

The Capstone Challenge of Defensive Engineering: In this final test, students review an entire production codebase exhibiting multiple interacting vulnerabilities.

Students must apply everything they have learned:

- Replace string-concatenated SQL queries with prepared statements.
- Escape all reflected and stored output strings.
- Enforce strict session-bound IDOR authorization matrices.
- Strip sensitive credentials from debug endpoints.
- Configure defensive HTTP headers and rate limiters.

When the automated test harness runs all attack vectors simultaneously and confirms that every exploit is blocked while the application still functions seamlessly, the student has earned the title of **Warden of the Keep**.

Epilogue: The Path Forward as an Ethical Guardian

Congratulations on completing your study of the Signals, Trading Post, and Forge dungeons!

The technical skills you have explored—socket binding, packet analysis, HTTP protocol mechanics, database injection, client-side script execution, and defensive engineering—are powerful tools. With that power comes a profound ethical responsibility.

The Ethics of the Craft:

The knowledge of how to pick a lock can be used by a burglar to rob a house in the night, or by a locksmith to rescue a child trapped in a room. The physical tools are identical; the intent and moral compass of the practitioner make all the difference.

Always practice your skills exclusively on systems you own or have explicit, documented, written permission to test (such as BreachKeep, approved CTFs, and educational labs). Channel your curiosity into building software that protects human privacy, defends vital infrastructure, and makes the digital world a safer place for everyone.

Quick Reference Toolkit

Tool / Command	Primary Use Case
<code>ss -ltn</code>	List local listening TCP sockets and ports without resolving names.
<code>nc -nv <ip> <port></code>	Connect to a raw TCP socket to grab banners or test communications.
<code>nmap -sT -p <ports> <ip></code>	Perform a TCP connect port scan across remote targets.
<code>tshark -r <file.pcap></code>	Analyze recorded packet capture streams from the command line.
<code>curl -i -H "... " <url></code>	Craft custom HTTP requests with injected headers and view response metadata.
<code>dig @<server> <domain> TXT</code>	Query DNS servers for specific record types and hidden text annotations.

— BreachKeep Faculty Curriculum • Designed for Curious Minds • Keep Moving Forward —